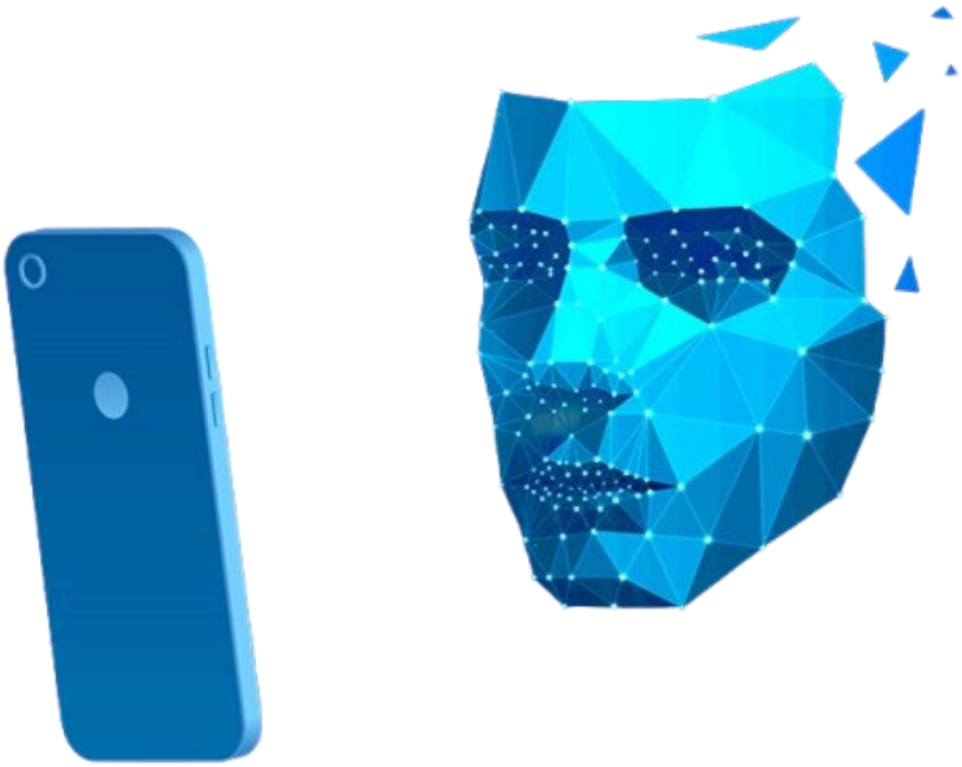
Reconocimiento Facial



Sandro Pegoraro Manzanedo

2025-2026

2 DAW

Contenido

Configuración del Entorno y Docker 3
Lógica del Backend y Microservicio 4
Interfaz Frontend 5
Despliegue y Ejecución 5
NGROK 6
Capturas de Funcionamiento 7
Correcto: 7
Incorrecto: 8

Configuración del Entorno y Docker

Modificación de los límites de subida en la configuración de PHP para evitar que el servidor bloquee las imágenes de alta resolución de la webcam

```
; Maximum allowed size for
; https://php.net/upload-ma
upload_max_filesize=20M
```

Sandro Pegoraro

```
; its value may be 0 to disable
; is disabled through enable_pos
; https://php.net/post-max-size
post_max_size=25M
```

Sandro Pegoraro

Configuración de la infraestructura Docker, estableciendo un mapeo de puertos específico para evitar colisiones con servicios locales del sistema operativo.

docker-compose.yml

```yaml
version: '3.8'
services:
  facial-api:
    build: ./microservicio_facial
    container_name: facial_service
    ports:
      - "8181:5000"
    restart: always
  db:
    image: postgres:15-alpine
    container_name: postgres_db
    ports:
      - "5433:5432"
    environment:
      POSTGRES_DB: plataforma_juegos
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: secretpassword
    volumes:
      - pgdata:/var/lib/postgresql/data
volumes:
  pgdata:
```

Sandro Pegoraro

Lógica del Backend y Microservicio

Lógica del microservicio en Python usando FastAPI. Se utiliza la librería DeepFace ajustando el parámetro de detección para evitar errores 500 por mala iluminación.

```python
@app.post("/verify")
async def verify_img(img1: UploadFile = File(...), img2: UploadFile = File(...)):
    path1 = f"temp_{img1.filename}"
    path2 = f"temp_{img2.filename}"

    try:

        with open(path1, "wb") as buffer:
            shutil.copyfileobj(img1.file, buffer)
        with open(path2, "wb") as buffer:
            shutil.copyfileobj(img2.file, buffer)


        result = DeepFace.verify(
            img1_path=path1,
            img2_path=path2,
            model_name="Facenet",
            detector_backend="opencv",
            enforce_detection=False
        )
```

Do you want to in

Sandro Pegoraro

El controlador de Laravel actuando como puente: recibe los archivos del formulario, los adjunta mediante HTTP Client y captura la respuesta JSON de Docker

```php
    $url = env('FACIAL_SERVICE_URL', 'http://127.0.0.1:8181/verify');

try {
        $response = Http::timeout(60)
            ->attach('img1', $request->file('foto_registro')->get(), 'reg.jpg')
            ->attach('img2', $request->file('foto_webcam')->get(), 'web.jpg')
            ->post($url);

        if (!$response->successful()) {
            return back()->withErrors(['El microservicio de IA ha fallado. Detalles: '
        }

        $datosPython = $response->json();
        return view('test-facial', ['resultado' => $datosPython]);
```

Sandro Pegoraro

Interfaz Frontend

Integración de WebRTC mediante JavaScript. Se captura el flujo de vídeo, se congela un fotograma usando un elemento <canvas> y se inyecta como blob

```
            <div class="error">❌ Identidad NO verificada (No coinciden).</div>
        @endif

        <p><strong>Nivel de diferencia (Distancia):</strong> {{ $resultado['distance' ?? 'N/A
        <p><strong>Modelo usado:</strong> {{ $resultado['model'] ?? 'N/A' }}</p>
    </div>
@endif

<video id="video" width="640" height="480" autoplay></video>
<canvas id="canvas" width="640" height="480" style="display:none;"></canvas>

<form id="formulario-facial" method="POST" action="/test-facial" enctype="multipart/form-data"
    @csrf
    <div style="margin-bottom: 15px;">
        <label><strong>1. Sube tu foto de registro (DNI/Perfil):</strong></label><br>
        <input type="file" name="foto_registro" required>
    </div>

    <input type="file" id="foto_webcam" name="foto_webcam" style="display:none;">

    <button type="button" id="btn-capturar" style="padding: 10px 20px; font-size: 1.1em; curso
        Capturar y Verificar
    </button>
</form>
```

Sandro Pegoraro

Despliegue y Ejecución

Los contenedores de la base de datos y la Inteligencia Artificial compilados y ejecutándose correctamente de fondo podemos verlo con el comando Docker PS

```
 => resolving provenance for metadata file                                    0.0s
[+] Running 3/3
 ✓ aplicacion-facial-facial-api  Built                                        0.0s
 ✓ Container facial_service      Started                                      0.6s
 ✓ Container postgres_db         Started                                      0.4s
PS E:\git-repos\Aplicacion-Facial-Documentacion\aplicacion-facial> docker ps
CONTAINER ID   IMAGE                          COMMAND                  CREATED          STATUS         PORTS
                                   NAMES
8f9588cc6241   aplicacion-facial-facial-api   "uvicorn main:app --…"   3 seconds ago    Up 3 seconds   0.0.0.0:81
81->5000/tcp, [::]:8181->5000/tcp   facial_service
868e3e36e1f4   postgres:15-alpine             "docker-entrypoint.s…"   About an hour ago   Up 3 seconds   0.0.0.0:54
33->5432/tcp, [::]:5433->5432/tcp   postgres_db
PS E:\git-repos\Aplicacion-Facial-Documentacion\aplicacion-facial>
```

Sandro Pegoraro

Servidor de desarrollo de Laravel activo y a la escucha de peticiones del cliente mediante el comando php artisan serve

```
CONTAINER ID   IMAGE                          COMMAND                  CREATED          STATUS          PORTS
                                   NAMES
8f9588cc6241   aplicacion-facial-facial-api   "uvicorn main:app --…"   3 seconds ago    Up 3 seconds    0.0.0.0:81
81->5000/tcp, [::]:8181->5000/tcp   facial_service
868e3e36e1f4   postgres:15-alpine             "docker-entrypoint.s…"   About an hour ago   Up 3 seconds   0.0.0.0:54
33->5432/tcp, [::]:5433->5432/tcp   postgres_db
PS E:\git-repos\Aplicacion-Facial-Documentacion\aplicacion-facial> php artisan serve

  INFO  Server running on [http://127.0.0.1:8000].

  Press Ctrl+C to stop the server
```

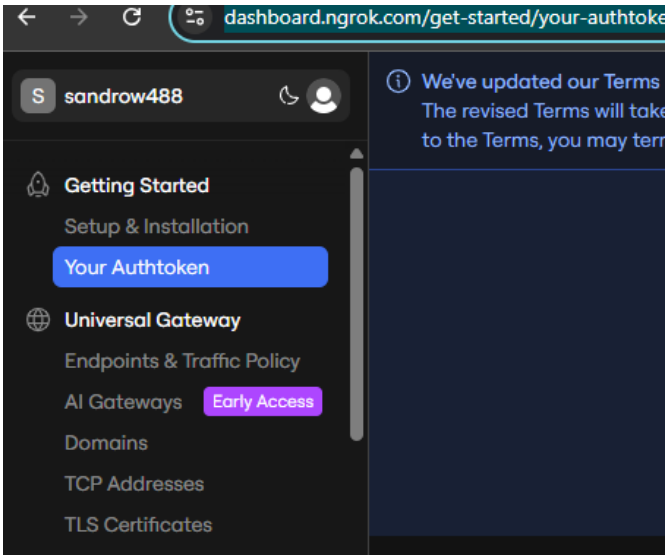
Sandro Pegoraro

NGROK

Buscaremos por Google del dispositivo local NGROK

Enlace de la web:
https://dashboard.ngrok.com

Iniciaremos sesión con GitHub o Google y en el panel de la izquierda buscaremos el apartado de tu authtoken



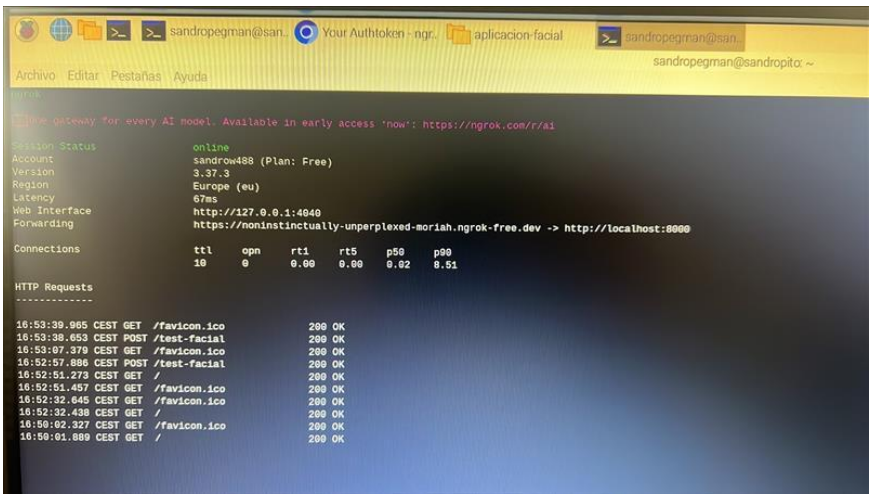
Dentro de este apartado encontraremos este token

3C7o681QhcdJI2R9oupitTCCpEt_3SXDxXTmLiKWVT2hvJFGi

Con este toquen lo utilizaremos con este comando:

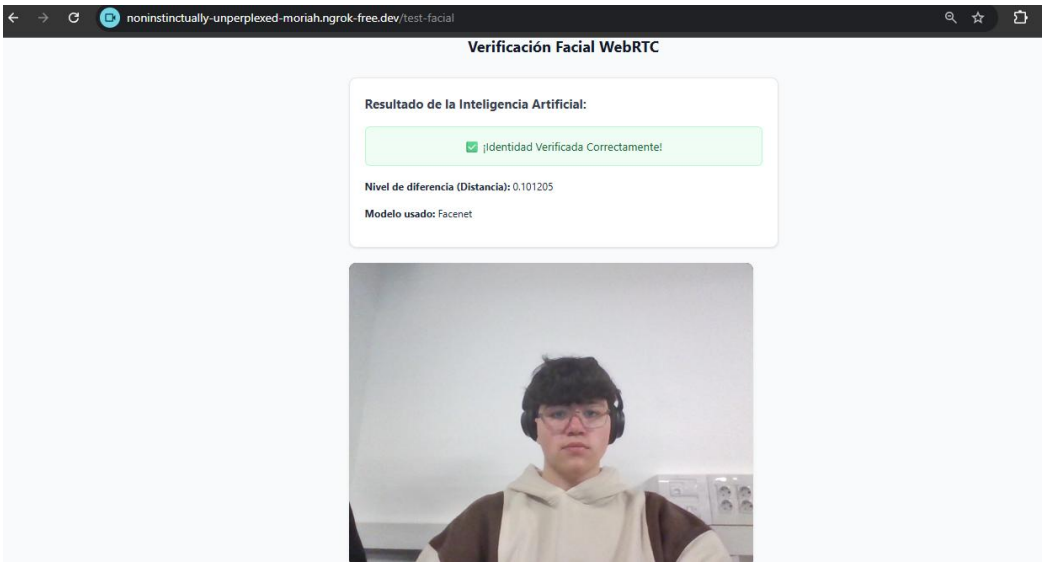
ngrok config add-authtoken 3C7o681QhcdJI2R9oupitTCCpEt_3SXDxXTmLiKW

Ahora lo que haremos es utilizar la url que nos dan con el comando que será esta: https://noninstinctually-unperplexed-moriah.ngrok-free.dev/test-facial

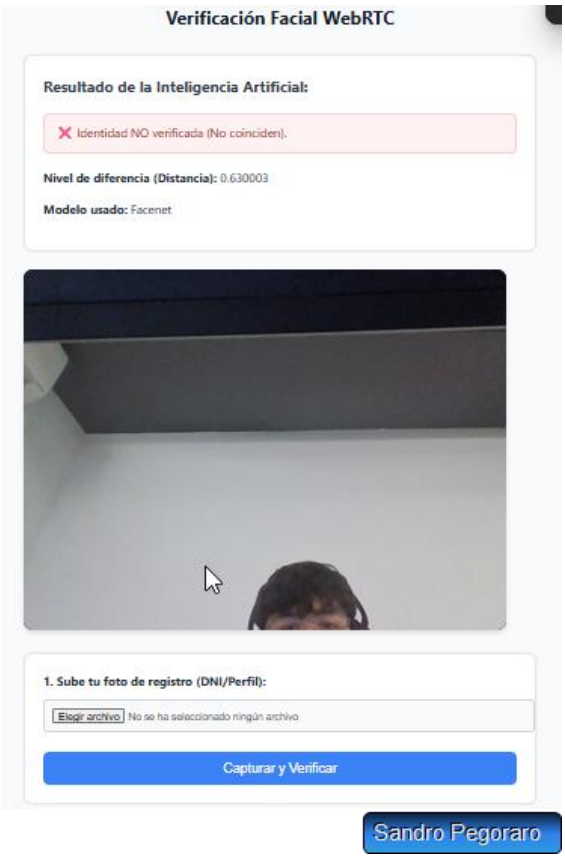


Capturas de Funcionamiento

Correcto:



Incorrecto:


Verificación Facial WebRTC
Resultado de la Inteligencia Artificial:
Identidad NO verificada (No coinciden).
Nivel de diferencia (Distancia): 0.630003
Modelo usado: Facenet
1. Sube tu foto de registro (DNI/Perfil):
Elegir archivo
No se ha seleccionado ningún archivo
Capturar y Verificar
Sandro Pegoraro